



Data Structures and Operations Using C++ in C Style

A Comprehensive 3-Day Intensive Training Guide

For Absolute Beginners → Expert Mastery

CHAPTER 9: DATA STRUCTURES AND OPERATIONS

Day 1 → Foundations & Arrays (9.0 - 9.1)

Day 2 → Linked Lists & Stacks (9.2 - 9.3)

Day 3 → Queues & Mastery (9.4)



Table of Contents

- [9.0 Data Structures and Operations](#)
- [9.1 Overview of Data Structures](#)
 - [Classification: Linear vs Non-Linear](#)
 - [Static Data Structures — Arrays](#)
 - [Dynamic Data Structures](#)
 - [Non-Linear Data Structures](#)

- [9.2 Implementing Linked List in C++](#)
- [9.3 Implementing Stacks in C++](#)
- [9.4 Implementing Queues in C++](#)

9.0 Data Structures and Operations

Learning Objectives

By the end of this chapter, you will be able to:

- Explain what a data structure is and why it matters in programming
- Distinguish between different categories of data structures
- Choose the right data structure for a given problem
- Implement core data structures from scratch in C++ using C-style patterns
- Analyze the time and space complexity of basic operations

Introduction: Why Do Data Structures Matter?

Imagine you are moving houses. You have hundreds of boxes. If you just throw everything in randomly, finding your coffee mug on moving day becomes a nightmare. But if you label each box by room and stack them in an organized way, you know exactly where to look.

Data structures are how computers organize data — and the choice of how you organize data can be the difference between a program that runs in 1 second versus one that takes 10 hours for the same task.

As a complete beginner, you may have used variables like `int x = 5;` to store a single value. But what if you need to store 10,000 student grades? Or model a social network with millions of friendships? That's where data structures come in.

In this guide, we write C++ code in **C style** — meaning we avoid modern C++ abstractions like `std::vector` or `std::list` and instead use raw pointers, arrays, and manual memory management. This teaches you exactly what is happening under the hood, giving you a deep, unshakeable understanding.

 **Pro Tip:** Understanding C-style data structures is the single best investment you can make as a programmer. Every modern language's standard library is built on exactly these concepts. When you know the internals, debugging becomes intuitive.

What Is a Data Structure?

DATA STRUCTURE

A way of organizing, storing, and managing data in a computer so that it can be accessed and modified efficiently.

DATA + RELATIONSHIPS + OPERATIONS = DS

Example: A phone book

- Data: names and phone numbers
- Relationship: names sorted alphabetically
- Operations: look up, add, remove entries

Every data structure is evaluated on two key metrics:

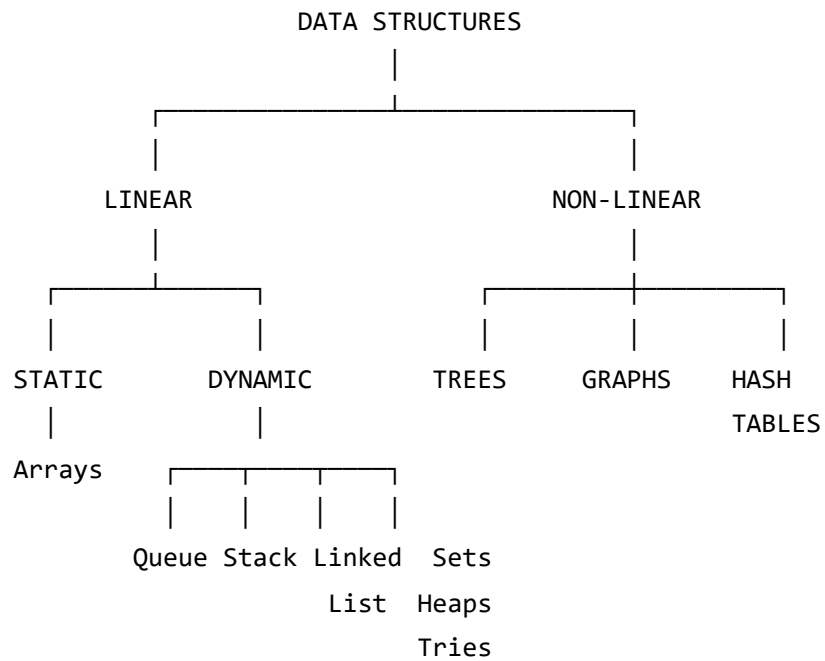
Metric	Question it Answers
Time Complexity	How fast can we perform operations?
Space Complexity	How much memory does it use?

We express these using **Big-O notation** — a simple shorthand:

- $O(1)$ → Constant time. Lightning fast. Always takes the same time.
- $O(\log n)$ → Logarithmic. Very fast. Doubles the data? Just 1 extra step.
- $O(n)$ → Linear. Fair. 10x data = 10x time.
- $O(n^2)$ → Quadratic. Slow for large data. 10x data = 100x time.

9.1 Overview of Data Structures

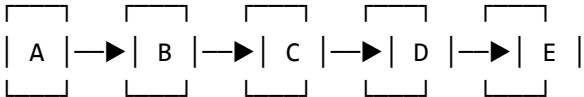
Classification: Linear vs Non-Linear



Linear Data Structures

In a **linear** data structure, elements are arranged sequentially — one after another, like beads on a string. Each element has exactly one predecessor (except the first) and one successor (except the last).

LINEAR (Sequential):

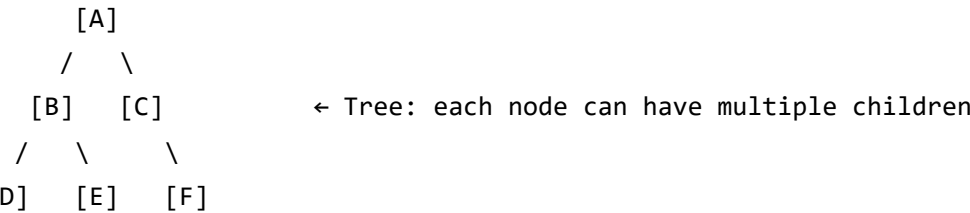


Each element connected to the next in sequence.

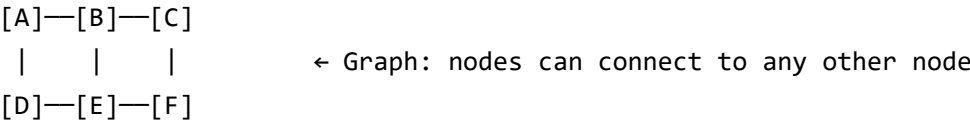
Non-Linear Data Structures

In a **non-linear** data structure, elements can be connected to multiple others — think of a family tree or a road map.

NON-LINEAR (Hierarchical / Network):



OR



Comparison Table: Linear vs Non-Linear

Feature	Linear	Non-Linear
Element arrangement	Sequential	Hierarchical / Network

Feature	Linear	Non-Linear
Memory use	Simple	More complex
Traversal	Single run	Multiple paths
Examples	Array, Stack, Queue, Linked List	Tree, Graph, Heap
Best for	Ordered data, FIFO/LIFO	Hierarchical data, Networks

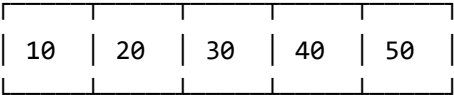
Static Data Structures — Arrays

What Is an Array?

An array is the simplest and most fundamental data structure. It stores elements of the **same type** in **contiguous (adjacent) memory locations**.

```
ARRAY: int arr[5] = {10, 20, 30, 40, 50};
```

Memory Address: 1000 1004 1008 1012 1016



Index: [0] [1] [2] [3] [4]

Each int takes 4 bytes. Elements stored back-to-back.

**Key Insight:** Because elements are in contiguous memory, accessing any element by index is **O(1)** — instant! The computer calculates:
`address = base_address + (index × element_size) .`

Why "Static"?

Arrays are **static** because their size is fixed at the time of creation. You must declare how many elements you need upfront.

```
int grades[30];    // Fixed at 30. Cannot grow or shrink.
```


Array Operations — C++ Code Example

```
#include <stdio.h>    // For printf (C-style output)

int main() {
    // — DECLARATION & INITIALIZATION —————
    int arr[5] = {10, 20, 30, 40, 50}; // Array of 5 integers
    int size = 5;

    // — ACCESS (O(1)) —————
    printf("First element: %d\n", arr[0]); // Output: 10
    printf("Third element: %d\n", arr[2]); // Output: 30
    printf("Last element:  %d\n", arr[4]); // Output: 50

    // — TRAVERSAL (O(n)) —————
    printf("\nAll elements: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);           // Output: 10 20 30 40 50
    }
    printf("\n");

    // — UPDATE (O(1)) —————
    arr[2] = 99;                          // Change index 2 from 30 to 99
    printf("After update: arr[2] = %d\n", arr[2]); // Output: 99

    // — SEARCH (O(n)) —————
    int target = 40;
    int found = -1;
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) {
            found = i;
        }
    }
}
```

```
        break;
    }
}
if (found != -1)
    printf("Found %d at index %d\n", target, found);
else
    printf("%d not found\n", target);

return 0;
}
```

Array Complexity Summary

ARRAY COMPLEXITY	
Operation	Time Complexity
Access	$O(1)$ ← BEST!
Search	$O(n)$
Insert	$O(n)$ (shift needed)
Delete	$O(n)$ (shift needed)
Space	$O(n)$

Real-World Use Cases for Arrays

- Storing a list of student grades

- Image pixels (a 2D array of color values)
- Lookup tables in embedded systems
- Storing game board state (chess, tic-tac-toe)

Common Mistakes with Arrays

✗ MISTAKE 1: Array Index Out of Bounds

```
int arr[5];  
arr[5] = 10;    // ERROR! Valid indices are 0-4 only.  
                // This causes undefined behavior (crash or corruption).
```

✗ MISTAKE 2: Using Uninitialized Array

```
int arr[5];  
printf("%d", arr[0]); // GARBAGE VALUE! Always initialize.
```

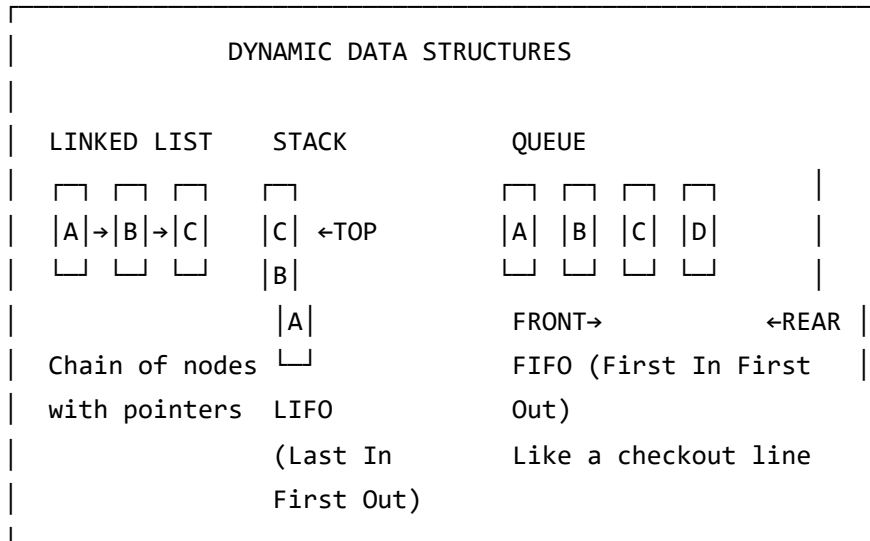
✗ MISTAKE 3: Wrong Size Calculation

```
int arr[5];  
int size = sizeof(arr);           // WRONG: gives bytes (20), not elements  
int size = sizeof(arr)/sizeof(arr[0]); // CORRECT: gives 5
```

Dynamic Data Structures

"Dynamic" means the structure can **grow or shrink at runtime** — you don't have to declare the size upfront. This is achieved using **dynamic memory allocation** (`malloc` / `free` in C, or `new` / `delete` in C++).

The three core dynamic linear data structures are:



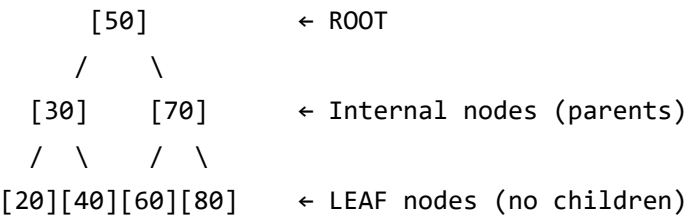
We will implement each of these in detail in sections 9.2, 9.3, and 9.4.

Non-Linear Data Structures

These are introduced here conceptually — full implementations are advanced topics.

Trees

A Tree has a ROOT node, PARENT nodes, CHILD nodes, and LEAF nodes.



Binary Search Tree (BST):

- Left child < Parent
- Right child > Parent
- Search: $O(\log n)$ on average

Tree Type	Use Case
Binary Search Tree	Fast search, insert, delete
AVL Tree	Self-balancing BST (always $O(\log n)$)
Heap	Priority queues, sorting
Trie	Autocomplete, spell checking
B-Tree	Database indexing

Graphs

Graphs model NETWORKS – any set of things with connections.

Undirected Graph:

A — B — C
| | |
D — E — F

Directed Graph (Digraph):

A → B → C
↑ ↓
F ← E ← D

Use cases:

- Social networks (Facebook friends)
- Road maps (Google Maps)
- Web pages (hyperlinks)
- Dependencies (build systems)

Hash Tables

A Hash Table maps KEYS to VALUES using a HASH FUNCTION.

Key: "Alice" → hash("Alice") = 3 → slots[3] = "555-1234"

Key: "Bob" → hash("Bob") = 7 → slots[7] = "555-5678"

0			
1			
2			
3	"Alice"	"555-1234"	← O(1) lookup!
4			
7	"Bob"	"555-5678"	

Average: O(1) for insert, delete, search – VERY fast!

Sets, Heaps, and Tries

SET: Stores unique values. No duplicates.

{1, 2, 3, 4, 5} (cannot contain two 3s)

Use: Eliminating duplicates, membership testing.

HEAP: A complete binary tree where parent $\geq$ children (Max-Heap).

Always gives you the largest (or smallest) element in $O(1)$.

Use: Priority queues, scheduling.

TRIE: A tree where each path from root to leaf spells a word.

Root

|— c — a — t ("cat")

| |— r ("car")

|— d — o — g ("dog")

Use: Autocomplete, dictionary, IP routing.



Section 9.1 — Exercises

Exercise 1 (Beginner): Write a C++ program that stores 5 temperatures in an array, then prints the highest and lowest values.

Exercise 2 (Intermediate): Given an array {5, 3, 8, 1, 9, 2, 7}, write a program to find and print all elements greater than 5.

Exercise 3 (Advanced): Implement a function that rotates an array to the right by k positions without using an extra array.

►  Solution Hints (Click to expand)



Section 9.1 Quick Reference

DATA STRUCTURES CHEAT SHEET			
Structure	Access	Search	Insert/Delete
Array	$O(1)$	$O(n)$	$O(n)$
Linked List	$O(n)$	$O(n)$	$O(1)$ at head
Stack	$O(n)$	$O(n)$	$O(1)$ push/pop
Queue	$O(n)$	$O(n)$	$O(1)$ enq/deq
Hash Table	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
BST	$O(\log n)$	$O(\log n)$	$O(\log n)$

9.2 Implementing Linked List in C++



Learning Objectives

- Understand what a node is and how pointers link them
- Distinguish between singly, doubly, and circular linked lists
- Implement all six core operations: insert at beginning, insert at end, insert at position, delete, update, and display
- Understand when to prefer linked lists over arrays

Introduction: The Problem with Arrays

Arrays are powerful, but they have one crippling weakness: **fixed size and expensive insertions/deletions**.

Inserting "25" at index 1 in an array requires shifting:

BEFORE: [10, 30, 40, 50, 60]

↑ Insert 25 here

SHIFT: [10, 30, 40, 50, 60] → move 30 right

[10, __, 30, 40, 50, 60] (need bigger array too!)

AFTER: [10, 25, 30, 40, 50, 60]

Every element after the insert point must MOVE.

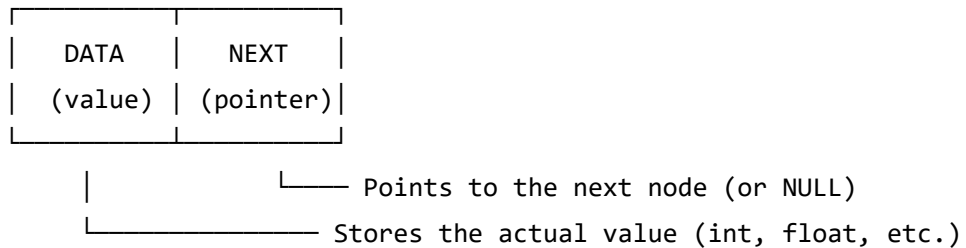
For 1,000,000 elements, that's up to 1,000,000 moves for ONE insert!

Linked lists solve this by storing elements as separate nodes connected by pointers. Adding or removing a node is just a matter of changing a pointer — no shifting required.

Definition and Types

What is a Node?

A NODE is the fundamental building block of a linked list.
It contains TWO parts:

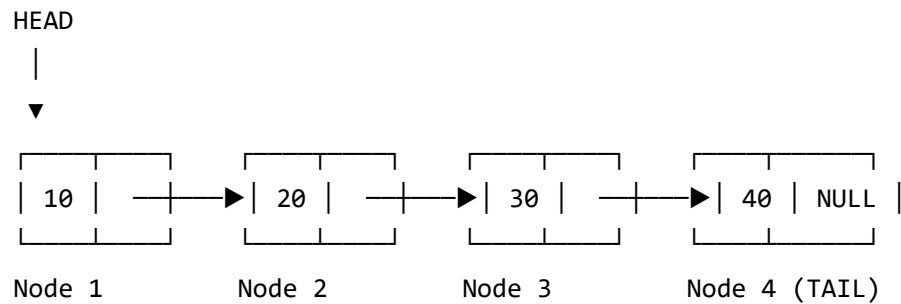


In C++ (C-style), we define this as a struct :

```
// A Node in a Singly Linked List
struct Node {
    int data;           // The value stored in this node
    Node* next;         // Pointer to the next node
};
```

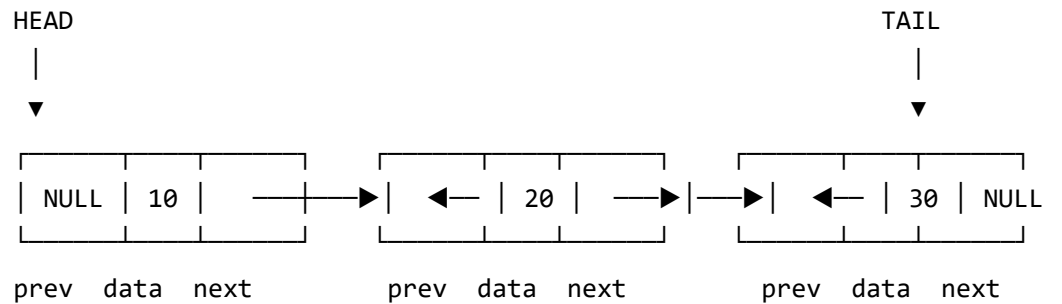
Type 1: Singly Linked List

Each node points to the **next** node only. You can only traverse forward.



Type 2: Doubly Linked List

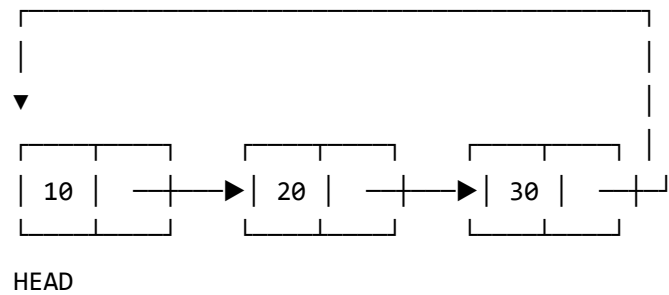
Each node points to both the **next** AND **previous** node. You can traverse in both directions.



```
// Doubly Linked List Node
struct DNode {
    int data;
    DNode* next;    // Points to next node
    DNode* prev;    // Points to previous node
};
```

Type 3: Circular Linked List

The **last node's** next **pointer** points back to the **first node** (HEAD), forming a circle.



No NULL at the end – it wraps around!
Useful for: Round-robin scheduling, circular buffers, music playlists.

Comparison Table

Feature	Singly	Doubly	Circular
Memory per node	Less (1 pointer)	More (2 pointers)	Same as Singly
Traversal	Forward only	Both directions	Continuous loop
Insert at beginning	O(1)	O(1)	O(1)
Insert at end	O(n)	O(1) with tail ptr	O(1) with tail ptr
Delete (given node)	O(n)	O(1)	O(n)
Use case	Simple list	Browser history	Scheduling

Complete Singly Linked List Implementation

Let's build a complete, working singly linked list with all operations.

Step 1: Node Structure and Global Head

```
#include <stdio.h>    // printf
#include <stdlib.h>    // malloc, free

// ——— NODE STRUCTURE —————
struct Node {
    int data;          // The value stored
    Node* next;        // Pointer to the next node (NULL if last)
};

// Global head pointer – points to the FIRST node in the list
// NULL means the list is empty
Node* head = NULL;
```

Step 2: Insert at Beginning — $O(1)$

BEFORE: HEAD $\rightarrow$ [20] $\rightarrow$ [30] $\rightarrow$ [40] $\rightarrow$ NULL

INSERT 10 at beginning:

Step 1: Create new node [10]

Step 2: new_node->next = head $\rightarrow$ [10] $\rightarrow$ [20] $\rightarrow$ [30] $\rightarrow$ [40] $\rightarrow$ NULL

Step 3: head = new_node $\rightarrow$ HEAD now points to [10]

AFTER: HEAD $\rightarrow$ [10] $\rightarrow$ [20] $\rightarrow$ [30] $\rightarrow$ [40] $\rightarrow$ NULL

```
// Insert a new node at the BEGINNING of the list
// Time: O(1) – constant time, no matter how big the list
void insertAtBeginning(int value) {
    // Step 1: Allocate memory for the new node
    Node* newNode = (Node*)malloc(sizeof(Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }

    // Step 2: Set the data
    newNode->data = value;

    // Step 3: New node's next = current head (could be NULL)
    newNode->next = head;

    // Step 4: Move head to point to the new node
    head = newNode;

    printf("Inserted %d at beginning.\n", value);
}
```


Step 3: Insert at End — $O(n)$

BEFORE: HEAD $\rightarrow$ [10] $\rightarrow$ [20] $\rightarrow$ [30] $\rightarrow$ NULL

INSERT 40 at end:

Step 1: Create new node [40], its next = NULL

Step 2: Traverse to the LAST node (the one with next == NULL)

Step 3: last_node->next = new_node

AFTER: HEAD $\rightarrow$ [10] $\rightarrow$ [20] $\rightarrow$ [30] $\rightarrow$ [40] $\rightarrow$ NULL

```
// Insert a new node at the END of the list
// Time: O(n) – we must traverse to find the last node
void insertAtEnd(int value) {
    // Step 1: Create the new node
    Node* newNode = (Node*)malloc(sizeof(Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }

    newNode->data = value;
    newNode->next = NULL;    // This will be the last node

    // Step 2: If list is empty, new node becomes the head
    if (head == NULL) {
        head = newNode;
        printf("Inserted %d (list was empty).\n", value);
        return;
    }

    // Step 3: Traverse to the last node
    Node* temp = head;
    while (temp->next != NULL) {    // Keep going until next is NULL
        temp = temp->next;
    }
    // Now temp points to the LAST node

    // Step 4: Link the last node to the new node
    temp->next = newNode;
```

```
    printf("Inserted %d at end.\n", value);  
}
```

Step 4: Insert at a Specific Position — $O(n)$

BEFORE: HEAD → [10] → [20] → [40] → NULL

INSERT 30 at position 2 (0-indexed, so between [20] and [40]):

Step 1: Create [30]

Step 2: Traverse to position 1 (the node BEFORE insertion point): [20]

Step 3: new_node->next = current_node->next → [30] → [40]

Step 4: current_node->next = new_node → [20] → [30]

AFTER: HEAD → [10] → [20] → [30] → [40] → NULL

```
// Insert a new node at a specific POSITION (0-indexed)
// Time: O(n) – must traverse to the position
void insertAtPosition(int value, int position) {
    // Special case: insert at beginning
    if (position == 0) {
        insertAtBeginning(value);
        return;
    }

    // Create new node
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = value;

    // Traverse to the node just BEFORE the desired position
    Node* temp = head;
    for (int i = 0; i < position - 1; i++) {
        if (temp == NULL) {
            printf("Position %d is out of range!\n", position);
            free(newNode);    // Don't leak memory!
            return;
        }
        temp = temp->next;
    }

    // Insert: link newNode between temp and temp->next
    newNode->next = temp->next;
    temp->next = newNode;
}
```

```
    printf("Inserted %d at position %d.\n", value, position);  
}
```

Step 5: Delete a Node — $O(n)$

BEFORE: HEAD → [10] → [20] → [30] → [40] → NULL

DELETE node with value 20:

Step 1: Find the node [20] AND the node BEFORE it ([10])

Step 2: prev->next = target->next → [10] → [30]

Step 3: free(target) → [20] is deallocated

AFTER: HEAD → [10] → [30] → [40] → NULL

```
// Delete the FIRST node with a matching value
// Time: O(n) – must search for the node
void deleteNode(int value) {
    if (head == NULL) {
        printf("List is empty. Nothing to delete.\n");
        return;
    }

    // Special case: delete the HEAD node
    if (head->data == value) {
        Node* temp = head;
        head = head->next;    // Move head forward
        free(temp);           // Free old head's memory
        printf("Deleted %d from beginning.\n", value);
        return;
    }

    // General case: find the node to delete
    Node* prev = head;        // Keep track of previous node
    Node* curr = head->next;   // Start from second node

    while (curr != NULL) {
        if (curr->data == value) {
            // Found it! Bypass curr in the chain
            prev->next = curr->next;
            free(curr);        // Free the deleted node's memory
            printf("Deleted %d.\n", value);
            return;
        }
        prev = curr;          // Advance both pointers
        curr = curr->next;
    }
}
```

```
    }

    // If we reach here, value wasn't found
    printf("Value %d not found in list.\n", value);
}
```

Step 6: Update a Node — $O(n)$

```
// Update the FIRST node with oldValue, replacing with newValue
// Time:  $O(n)$ 
void updateNode(int oldValue, int newValue) {
    Node* temp = head;

    while (temp != NULL) {
        if (temp->data == oldValue) {
            temp->data = newValue;    // Update in place
            printf("Updated %d to %d.\n", oldValue, newValue);
            return;
        }
        temp = temp->next;
    }

    printf("Value %d not found.\n", oldValue);
}
```

Step 7: Display the List — $O(n)$

```
// Print all elements in the list
// Time:  $O(n)$ 
void displayList() {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    printf("List: HEAD → ");
    Node* temp = head;

    while (temp != NULL) {
        printf("[%d] → ", temp->data);
        temp = temp->next;
    }

    printf("NULL\n");
}
```


Step 8: Complete Program — Putting It All Together

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    Node* next;
};

Node* head = NULL;

// ... (all functions from above) ...

int main() {
    printf("=== Singly Linked List Demo ===\n\n");

    // Build list: 10 → 30 → 40
    insertAtEnd(30);
    insertAtEnd(40);
    insertAtBeginning(10);
    displayList();    // HEAD → [10] → [30] → [40] → NULL

    // Insert 20 at position 1
    insertAtPosition(20, 1);
    displayList();    // HEAD → [10] → [20] → [30] → [40] → NULL

    // Update 30 to 99
    updateNode(30, 99);
    displayList();    // HEAD → [10] → [20] → [99] → [40] → NULL
```

```
// Delete 20
deleteNode(20);
displayList(); // HEAD → [10] → [99] → [40] → NULL

return 0;
}
```

Expected Output:

=== Singly Linked List Demo ===

Inserted 30 at end.

Inserted 40 at end.

Inserted 10 at beginning.

List: HEAD → [10] → [30] → [40] → NULL

Inserted 20 at position 1.

List: HEAD → [10] → [20] → [30] → [40] → NULL

Updated 30 to 99.

List: HEAD → [10] → [20] → [99] → [40] → NULL

Deleted 20.

List: HEAD → [10] → [99] → [40] → NULL

Doubly Linked List — Key Operations

```
#include <stdio.h>
#include <stdlib.h>

struct DNode {
    int data;
    DNode* next;
    DNode* prev;
};

DNode* dhead = NULL;

// Insert at beginning of doubly linked list
void dInsertAtBeginning(int value) {
    DNode* newNode = (DNode*)malloc(sizeof(DNode));
    newNode->data = value;
    newNode->prev = NULL;           // New node has no previous
    newNode->next = dhead;         // New node points forward to old head

    if (dhead != NULL)
        dhead->prev = newNode;    // Old head's prev now points back to new node

    dhead = newNode;              // Update head
    printf("DLL: Inserted %d at beginning.\n", value);
}

// Display doubly linked list FORWARD
void dDisplayForward() {
    printf("Forward:  HEAD ⇄ ");
```

```
DNode* temp = dhead;
while (temp != NULL) {
    printf("[%d] ⇌ ", temp->data);
    temp = temp->next;
}
printf("NULL\n");
}

// Display doubly linked list BACKWARD (find tail first)
void dDisplayBackward() {
    if (dhead == NULL) { printf("List empty.\n"); return; }

    // Traverse to the tail
    DNode* temp = dhead;
    while (temp->next != NULL)
        temp = temp->next;

    // Now traverse backward using prev pointers
    printf("Backward: NULL ⇌ ");
    while (temp != NULL) {
        printf("[%d] ⇌ ", temp->data);
        temp = temp->prev;
    }
    printf("HEAD\n");
}

int main() {
    dInsertAtBeginning(30);
    dInsertAtBeginning(20);
    dInsertAtBeginning(10);
    dDisplayForward();    // HEAD ⇌ [10] ⇌ [20] ⇌ [30] ⇌ NULL
    dDisplayBackward();   // NULL ⇌ [30] ⇌ [20] ⇌ [10] ⇌ HEAD
}
```

```
    return 0;  
}
```

Real-World Use Cases for Linked Lists

LINKED LIST REAL-WORLD APPLICATIONS
• Web browser history (back/forward = doubly linked) • Music playlist (next song, previous song) • Undo/Redo in text editors • Operating system process queue • Memory allocator free-list • Hash table chaining (collision resolution)



Common Mistakes with Linked Lists

✗ MISTAKE 1: Forgetting to free memory (memory leak)

```
void deleteNode(int v) {  
    Node* temp = head;  
    head = head->next;  
    // WRONG: forgot free(temp)! Memory is now leaked.  
}
```

✗ MISTAKE 2: Dereferencing NULL pointer

```
Node* temp = head;  
while (temp->next != NULL) // CRASH if temp is NULL!  
// FIX: while (temp != NULL && temp->next != NULL)
```

✗ MISTAKE 3: Losing the head pointer

```
head = head->next; // If you don't save old head first, it's GONE!  
// FIX: Node* temp = head; head = head->next; free(temp);
```

✗ MISTAKE 4: Off-by-one in position insert

```
// "Insert at position 3" – does this mean the 3rd element  
// or after the 3rd element? Always clarify your indexing.
```



Best Practices for Linked Lists

- ✓ Always initialize pointers to NULL
- ✓ Always check for NULL before dereferencing
- ✓ Always free() every malloc() – pair them carefully
- ✓ Keep a tail pointer for O(1) insertions at end
- ✓ Write a freeList() function to clean up when done
- ✓ Consider using a sentinel (dummy) head node to simplify edge cases



Section 9.2 — Exercises

Exercise 1 (Beginner): Create a linked list with values 1, 2, 3, 4, 5 and display it.

Exercise 2 (Intermediate): Write a function `reverseList()` that reverses a singly linked list in place (without creating new nodes).

Exercise 3 (Advanced): Write a function `findMiddle()` that returns the middle element of a linked list in a single pass (without counting length first). Hint: Use two pointers — one slow (moves 1 step) and one fast (moves 2 steps).



Section 9.2 Quick Reference

LINKED LIST OPERATIONS CHEAT SHEET		
Operation	Singly LL	Doubly LL
Insert at head	$O(1)$	$O(1)$
Insert at tail	$O(n)$ / $O(1)^*$	$O(1)$ with tail ptr
Insert at pos	$O(n)$	$O(n)$
Delete at head	$O(1)$	$O(1)$
Delete by value	$O(n)$	$O(n)$
Search	$O(n)$	$O(n)$
Traverse fwd	Yes	Yes
Traverse bwd	No	Yes

* $O(1)$ at tail if you maintain a tail pointer

9.3 Implementing Stacks in C++



Learning Objectives

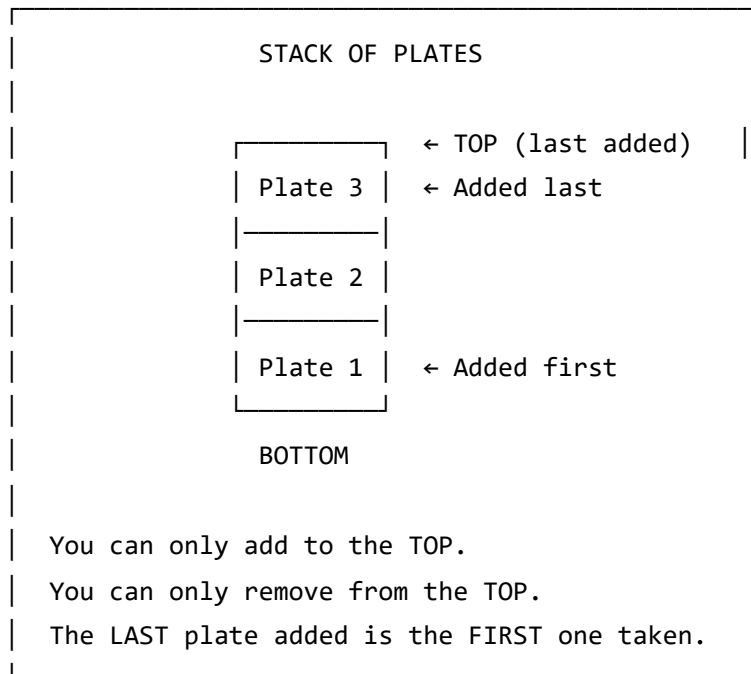
- Understand the LIFO principle and why it's powerful
- Implement a stack using both an array and a linked list
- Master the four core stack operations: Push, Pop, Peek, isEmpty

- Apply stacks to real-world problems

Introduction: The Last In, First Out Principle

A **stack** is one of the most elegant and useful data structures in computer science. It follows the **LIFO** rule: **L**ast **I**n, **F**irst **O**ut.

Think of a stack of plates at a buffet restaurant:



Real-World Stack Analogies

-  Stack of books → Can only take from the top
-  Browser Back btn → Go back in reverse order of pages visited
-  Ctrl+Z (Undo) → Undo last action first
-  Function calls → Return in reverse order of how they were called

Definition and Characteristics

STACK DEFINITION:

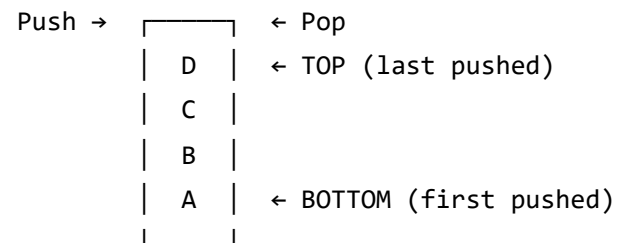
A linear data structure that follows LIFO order.

All insertions and deletions happen at ONE end: the TOP.

KEY PROPERTIES:

- Only the TOP element is accessible at any time
- Push adds to top
- Pop removes from top
- Peek reads the top without removing it
- Fixed (array-based) or dynamic (linked list-based) size

STACK VISUALIZATION:



Stack Using Array — Complete Implementation

An array-based stack uses a fixed-size array and a `top` variable to track the current top index.

```
#include <stdio.h>
#include <stdlib.h>

// ——— STACK CONFIGURATION ———
#define MAX_SIZE 100 // Maximum number of elements

// ——— STACK STRUCTURE ———
struct Stack {
    int data[MAX_SIZE]; // Array to store elements
    int top;             // Index of the top element (-1 if empty)
};

// ——— INITIALIZE STACK ———
// Must call this before using the stack!
void initStack(Stack* s) {
    s->top = -1; // -1 means the stack is empty
    printf("Stack initialized.\n");
}

// ——— IS EMPTY ———
// Returns 1 (true) if stack is empty, 0 (false) otherwise
// Time: O(1)
int isEmpty(Stack* s) {
    return s->top == -1;
}

// ——— IS FULL ———
// Returns 1 (true) if stack is full (array-based stacks have a limit)
// Time: O(1)
int isFull(Stack* s) {
    return s->top == MAX_SIZE - 1;
}
```

```
}

// ——— PUSH —————
// Add an element to the TOP of the stack
// Time: O(1)
void push(Stack* s, int value) {
    if (isFull(s)) {
        printf("Stack OVERFLOW! Cannot push %d – stack is full.\n", value);
        return;
    }

    s->top++;           // Move top up
    s->data[s->top] = value; // Store value at new top

    printf("Pushed: %d (top index = %d)\n", value, s->top);
}

// ——— POP —————
// Remove and return the TOP element
// Time: O(1)
int pop(Stack* s) {
    if (isEmpty(s)) {
        printf("Stack UNDERFLOW! Cannot pop – stack is empty.\n");
        return -1; // Sentinel value indicating error
    }

    int value = s->data[s->top]; // Save the top value
    s->top--;                    // Move top down (logically removes it)

    printf("Popped: %d (top index = %d)\n", value, s->top);
    return value;
}
```

```
// — PEEK —————  
// Look at the TOP element WITHOUT removing it  
// Time: O(1)  
int peek(Stack* s) {  
    if (isEmpty(s)) {  
        printf("Stack is empty – nothing to peek.\n");  
        return -1;  
    }  
  
    printf("Peek:  %d\n", s->data[s->top]);  
    return s->data[s->top];  
}  
  
// — DISPLAY —————  
// Show all elements (top to bottom)  
// Time: O(n)  
void displayStack(Stack* s) {  
    if (isEmpty(s)) {  
        printf("Stack is empty.\n");  
        return;  
    }  
  
    printf("Stack (top → bottom): [ ");  
    for (int i = s->top; i >= 0; i--) {  
        printf("%d ", s->data[i]);  
    }  
    printf("]\n");  
}  
  
// — MAIN DEMO —————  
int main() {
```

```
Stack myStack;
initStack(&myStack);

printf("\n--- Pushing elements ---\n");
push(&myStack, 10);
push(&myStack, 20);
push(&myStack, 30);
push(&myStack, 40);

printf("\n--- Current stack ---\n");
displayStack(&myStack);

printf("\n--- Peek at top ---\n");
peek(&myStack);

printf("\n--- Popping elements ---\n");
pop(&myStack);
pop(&myStack);

printf("\n--- Stack after pops ---\n");
displayStack(&myStack);

printf("\n--- isEmpty check ---\n");
printf("Stack empty? %s\n", isEmpty(&myStack) ? "YES" : "NO");

return 0;
}
```

Expected Output:

Stack initialized.

--- Pushing elements ---

Pushed: 10 (top index = 0)

Pushed: 20 (top index = 1)

Pushed: 30 (top index = 2)

Pushed: 40 (top index = 3)

--- Current stack ---

Stack (top → bottom): [40 30 20 10]

--- Peek at top ---

Peek: 40

--- Popping elements ---

Popped: 40 (top index = 2)

Popped: 30 (top index = 1)

--- Stack after pops ---

Stack (top → bottom): [20 10]

--- isEmpty check ---

Stack empty? NO

Stack Using Linked List — Dynamic Implementation

A linked list-based stack has **no size limit** (limited only by available memory). The `head` of the list acts as the top of the stack.


```
#include <stdio.h>
#include <stdlib.h>

// ——— NODE FOR LINKED LIST STACK ———
struct StackNode {
    int data;
    StackNode* next;
};

StackNode* stackTop = NULL; // Points to the top node (NULL = empty)

// ——— PUSH (LINKED LIST VERSION) ———
// Time: O(1)
void pushLL(int value) {
    StackNode* newNode = (StackNode*)malloc(sizeof(StackNode));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = value;
    newNode->next = stackTop; // New node points to current top
    stackTop = newNode;      // New node IS the new top
    printf("Pushed: %d\n", value);
}

// ——— POP (LINKED LIST VERSION) ———
// Time: O(1)
int popLL() {
    if (stackTop == NULL) {
        printf("Stack is empty!\n");
        return -1;
    }
}
```

```
    }  
    StackNode* temp = stackTop;  
    int value = stackTop->data;  
    stackTop = stackTop->next;    // Move top to next node  
    free(temp);                  // Free memory of old top  
    printf("Popped: %d\n", value);  
    return value;  
}
```

// — PEEK (LINKED LIST VERSION) —————

// Time: O(1)

```
int peekLL() {  
    if (stackTop == NULL) {  
        printf("Stack is empty!\n");  
        return -1;  
    }  
    printf("Peek: %d\n", stackTop->data);  
    return stackTop->data;  
}
```

// — IS EMPTY (LINKED LIST VERSION) —————

```
int isEmptyLL() {  
    return stackTop == NULL;  
}
```

Stack Complexity Summary

STACK COMPLEXITY		
Operation	Array Stack	Linked List Stack
Push	$O(1)$	$O(1)$
Pop	$O(1)$	$O(1)$
Peek	$O(1)$	$O(1)$
isEmpty	$O(1)$	$O(1)$
Space	$O(n)$ fixed	$O(n)$ dynamic
Overflow?	YES (MAX_SIZE)	NO (heap limited)

Practical Application: Balanced Parentheses Checker

One of the most classic stack applications — checking if brackets are balanced:

```
#include <stdio.h>
#include <string.h>

#define MAX 100

// Simplified stack just for characters
char charStack[MAX];
int charTop = -1;

void pushChar(char c) { charStack[++charTop] = c; }
char popChar()      { return charStack[charTop--]; }
int isEmptyChar()   { return charTop == -1; }

// Returns 1 if expression has balanced brackets, 0 otherwise
int isBalanced(const char* expr) {
    int len = strlen(expr);

    for (int i = 0; i < len; i++) {
        char ch = expr[i];

        // If opening bracket, push it
        if (ch == '(' || ch == '[' || ch == '{') {
            pushChar(ch);
        }
        // If closing bracket, check it matches the top
        else if (ch == ')' || ch == ']' || ch == '}') {
            if (isEmptyChar()) return 0;    // No matching opener

            char top = popChar();

            // Check if pairs match

```

```

        if ((ch == ')' && top != '(') ||
            (ch == ']' && top != '[') ||
            (ch == '}' && top != '{')) {
            return 0;    // Mismatched pair
        }
    }
}

return isEmptyChar();    // If stack is empty, all matched!
}

int main() {
    const char* tests[] = {
        "{[()]}",        // Balanced
        "{[( )]}",        // NOT balanced (wrong order)
        "((()))",        // Balanced
        "((())",          // NOT balanced (missing close)
        ""                // Empty = balanced
    };

    for (int i = 0; i < 5; i++) {
        charTop = -1;    // Reset stack for each test
        printf("%-12s → %s\n", tests[i],
            isBalanced(tests[i]) ? "BALANCED ✓" : "NOT BALANCED ✗");
    }
    return 0;
}

```

Output:

{[()] } → BALANCED ✓
{[()] } → NOT BALANCED ✗
(()) → BALANCED ✓
(()) → NOT BALANCED ✗
→ BALANCED ✓

Real-World Use Cases for Stacks

STACK REAL-WORLD APPLICATIONS
• Function call stack (every program uses this!) • Expression evaluation (calculators) • Syntax checking (compilers use stacks) • Undo/Redo functionality • Browser back/forward navigation • Depth-First Search (DFS) in graphs • Tower of Hanoi puzzle



Common Mistakes with Stacks

✗ **MISTAKE 1: Stack Overflow** – pushing to a full array stack
Always check `isFull()` before `push()` on array-based stacks.

✗ **MISTAKE 2: Stack Underflow** – popping from an empty stack
Always check `isEmpty()` before `pop()` or `peek()`.

✗ **MISTAKE 3: Forgetting the top index starts at -1**
`int top = 0; // WRONG – data[0] is now "occupied" conceptually`
`int top = -1; // CORRECT – -1 means empty, first push goes to [0]`

✗ **MISTAKE 4: Not initializing the stack**
`Stack s;`
`push(&s, 10); // UNDEFINED BEHAVIOR – s.top is garbage!`
`// Always call initStack() first.`



Section 9.3 — Exercises

Exercise 1 (Beginner): Push the numbers 5, 10, 15, 20 onto a stack, then pop them all and print each popped value. Notice the LIFO order.

Exercise 2 (Intermediate): Write a function `reverseString(char* str)` using a stack that reverses a string in place.

Exercise 3 (Advanced): Implement a function that evaluates a postfix expression (e.g., `"3 4 + 5 *" = (3+4)*5 = 35`) using a stack.



Section 9.3 Quick Reference

STACK OPERATIONS CHEAT SHEET		
Operation	What it does	Time
initStack(s)	Sets top = -1	O(1)
push(s, val)	Add val to top	O(1)
pop(s)	Remove & return top	O(1)
peek(s)	Return top (no removal)	O(1)
isEmpty(s)	Check if top == -1	O(1)
isFull(s)	Check if top == MAX-1	O(1)
KEY RULE: LIFO – Last In, First Out		
TOP is the only accessible end		

9.4 Implementing Queues in C++



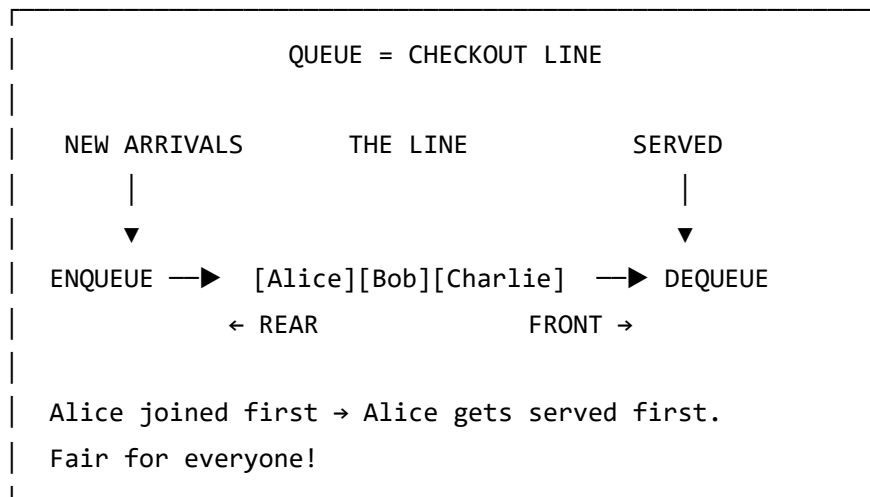
Learning Objectives

- Understand the FIFO principle and where it applies
- Distinguish between Simple, Circular, Priority, and Dequeue queue variants
- Implement all four core operations: Enqueue, Dequeue, Peek, isEmpty

- Understand and fix the "false full" problem in simple array queues

Introduction: First Come, First Served

A **queue** models the most natural and fair ordering in the world: **first come, first served**. It follows the **FIFO** rule: **F**irst **I**n, **F**irst **O**ut.



Types of Queues

Type 1: Simple (Linear) Queue

The basic queue. Elements enter from the REAR and exit from the FRONT.

10	20	30	40	50
----	----	----	----	----

FRONT REAR
 (dequeue here) (enqueue here)

Problem: After several enqueues and dequeues, the FRONT pointer moves right, wasting array space even if there's room behind FRONT!

After 3 dequeues and 3 enqueues:

			40	50	60	70	80
--	--	--	----	----	----	----	----

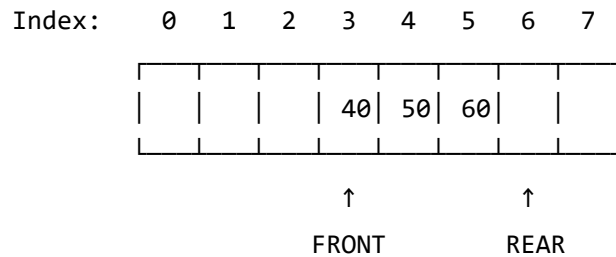
[WASTED SPACE] FRONT REAR

The array might report "full" even though indices 0-2 are free!
 This is the "false full" problem – solved by the circular queue.

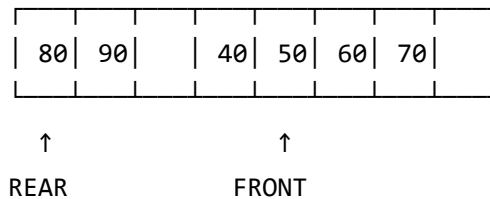
Type 2: Circular Queue

The REAR wraps around to the beginning of the array when it reaches the end. This eliminates wasted space.

CIRCULAR QUEUE (array of size 8):



When REAR reaches index 7 and wraps:



New elements fill the "wasted" front space!

Formula: $REAR = (REAR + 1) \% MAX_SIZE$

Type 3: Priority Queue

Elements are served based on **priority**, not arrival order. Higher priority = served first.

PRIORITY QUEUE:

Enqueue: (Task A, priority 2), (Task B, priority 5), (Task C, priority 1)

Internally sorted by priority:

Task B (P=5)	Task A (P=2)	Task C (P=1)
--------------	--------------	--------------

FRONT (served first)

REAR

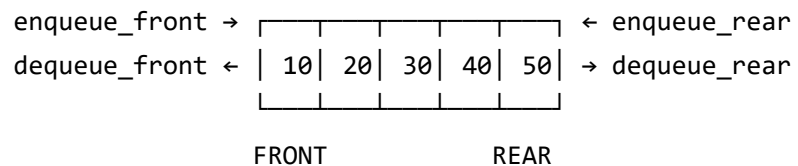
Dequeue gives Task B first (highest priority).

Use cases: CPU scheduling, emergency room triage, Dijkstra's algorithm.

Type 4: Double-Ended Queue (Deque)

Elements can be added or removed from **both ends**.

DEQUE:



Use cases: Sliding window algorithms, undo-redo with both ends.

Queue Type Comparison

Feature	Simple	Circular	Priority	Deque
Memory efficiency	✗ Wastes space	✓ Efficient	Depends	✓
Order	FIFO strict	FIFO strict	Priority-based	Both ends
Complexity (enq/deq)	O(1)	O(1)	O(n) or O(log n)	O(1)
Use case	Basic ordering	Buffer, streaming	Scheduling	Sliding window

Simple Queue — Array Implementation

```
#include <stdio.h>

#define MAX 6 // Queue can hold up to 6 elements

// — QUEUE STRUCTURE —
struct Queue {
    int data[MAX];
    int front; // Index of first element
    int rear; // Index of last element
    int count; // Number of elements currently in queue
};

// — INITIALIZE —
void initQueue(Queue* q) {
    q->front = 0;
    q->rear = -1;
    q->count = 0;
    printf("Queue initialized.\n");
}

// — IS EMPTY —
int qIsEmpty(Queue* q) {
    return q->count == 0;
}

// — IS FULL —
int qIsFull(Queue* q) {
    return q->count == MAX;
}
```

```
// — ENQUEUE — Add to REAR —————  
// Time: O(1)  
void enqueue(Queue* q, int value) {  
    if (qIsFull(q)) {  
        printf("Queue OVERFLOW! Cannot enqueue %d.\n", value);  
        return;  
    }  
    q->rear++;  
    q->data[q->rear] = value;  
    q->count++;  
    printf("Enqueued: %d\n", value);  
}  
  
// — DEQUEUE — Remove from FRONT —————  
// Time: O(1)  
int dequeue(Queue* q) {  
    if (qIsEmpty(q)) {  
        printf("Queue UNDERFLOW! Cannot dequeue — empty.\n");  
        return -1;  
    }  
    int value = q->data[q->front];  
    q->front++;  
    q->count--;  
    printf("Dequeued: %d\n", value);  
    return value;  
}  
  
// — PEEK — Look at FRONT without removing —————  
// Time: O(1)  
int qPeek(Queue* q) {  
    if (qIsEmpty(q)) {
```

```
        printf("Queue is empty!\n");
        return -1;
    }
    printf("Front element: %d\n", q->data[q->front]);
    return q->data[q->front];
}
```

```
// ——— DISPLAY ———
```

```
void displayQueue(Queue* q) {
    if (qIsEmpty(q)) {
        printf("Queue is empty.\n");
        return;
    }
    printf("Queue [FRONT → REAR]: ");
    for (int i = q->front; i <= q->rear; i++) {
        printf("%d ", q->data[i]);
    }
    printf("\n");
}
```

```
// ——— MAIN DEMO ———
```

```
int main() {
    Queue q;
    initQueue(&q);

    printf("\n--- Enqueue 4 elements ---\n");
    enqueue(&q, 10);
    enqueue(&q, 20);
    enqueue(&q, 30);
    enqueue(&q, 40);
    displayQueue(&q);
}
```



```
printf("\n--- Peek front ---\n");  
qPeek(&q);  
  
printf("\n--- Dequeue 2 elements ---\n");  
dequeue(&q);  
dequeue(&q);  
displayQueue(&q);  
  
printf("\n--- Enqueue 2 more ---\n");  
enqueue(&q, 50);  
enqueue(&q, 60);  
displayQueue(&q);  
  
return 0;  
}
```

Circular Queue — Complete Implementation

The circular queue fixes the false-full problem using modular arithmetic.

```
#include <stdio.h>
#define CMAX 5

struct CircularQueue {
    int data[CMAX];
    int front;
    int rear;
    int count;
};

void cqInit(CircularQueue* q) {
    q->front = 0;
    q->rear = -1;
    q->count = 0;
}

int cqIsEmpty(CircularQueue* q) { return q->count == 0; }
int cqIsFull(CircularQueue* q) { return q->count == CMAX; }

// ——— CIRCULAR ENQUEUE —————
// The KEY difference: use modulo to wrap rear around
// Time: O(1)
void cqEnqueue(CircularQueue* q, int value) {
    if (cqIsFull(q)) {
        printf("Circular Queue FULL! Cannot enqueue %d.\n", value);
        return;
    }
    // Wrap rear: if at end, go back to index 0
    q->rear = (q->rear + 1) % CMAX;
    q->data[q->rear] = value;
    q->count++;
}
```

```
    printf("CQ Enqueued: %d (rear at index %d)\n", value, q->rear);
}

// — CIRCULAR DEQUEUE —————
// Also wraps front using modulo
// Time: O(1)
int cqDequeue(CircularQueue* q) {
    if (cqIsEmpty(q)) {
        printf("Circular Queue EMPTY!\n");
        return -1;
    }
    int value = q->data[q->front];
    // Wrap front: if at end, go back to index 0
    q->front = (q->front + 1) % CMAX;
    q->count--;
    printf("CQ Dequeued: %d (front at index %d)\n", value, q->front);
    return value;
}

// — CIRCULAR DISPLAY —————
void cqDisplay(CircularQueue* q) {
    if (cqIsEmpty(q)) { printf("CQ: empty\n"); return; }

    printf("CQ [FRONT → REAR]: ");
    int i = q->front;
    for (int k = 0; k < q->count; k++) {
        printf("%d ", q->data[i]);
        i = (i + 1) % CMAX;    // Wrap around
    }
    printf("\n");
}
```

```
int main() {
    CircularQueue cq;
    cqInit(&cq);

    printf("=== Circular Queue Demo ===\n\n");

    cqEnqueue(&cq, 10);
    cqEnqueue(&cq, 20);
    cqEnqueue(&cq, 30);
    cqEnqueue(&cq, 40);
    cqEnqueue(&cq, 50);    // Queue is now full (CMAX = 5)
    cqDisplay(&cq);

    printf("\nDequeue 2 elements (frees space at front):\n");
    cqDequeue(&cq);
    cqDequeue(&cq);
    cqDisplay(&cq);

    printf("\nEnqueue 2 more (wraps around):\n");
    cqEnqueue(&cq, 60);
    cqEnqueue(&cq, 70);
    cqDisplay(&cq);

    return 0;
}
```

Key Visual — The Wrap-Around:

After full (indices 0-4 used):

Index:	0	1	2	3	4
	[10]	[20]	[30]	[40]	[50]
	↑			↑	
	FRONT			REAR	

After dequeue x2 (front moves to index 2):

Index:	0	1	2	3	4
	[]	[]	[30]	[40]	[50]
			↑	↑	
			FRONT	REAR	

After enqueue 60, 70 (rear WRAPS to indices 0 and 1!):

Index:	0	1	2	3	4
	[60]	[70]	[30]	[40]	[50]
	↑	↑			
	REAR	FRONT			

All 5 slots are used efficiently!

Priority Queue — Simple Array Implementation

```
#include <stdio.h>

#define PMAX 10

struct PQItem {
    int value;
    int priority;    // Higher number = higher priority
};

struct PriorityQueue {
    PQItem items[PMAX];
    int size;
};

void pqInit(PriorityQueue* pq) { pq->size = 0; }

// ——— PRIORITY ENQUEUE ———
// Insert in sorted order by priority (highest first)
// Time: O(n)
void pqEnqueue(PriorityQueue* pq, int value, int priority) {
    if (pq->size == PMAX) {
        printf("Priority Queue is full!\n");
        return;
    }

    // Find correct position (maintain sorted order)
    int i = pq->size - 1;
    while (i >= 0 && pq->items[i].priority < priority) {
        pq->items[i + 1] = pq->items[i];    // Shift lower-priority items
```

```
        i--;
    }
    pq->items[i + 1].value    = value;
    pq->items[i + 1].priority = priority;
    pq->size++;

    printf("PQ Enqueued: value=%d, priority=%d\n", value, priority);
}

// ——— PRIORITY DEQUEUE ———
// Remove and return the HIGHEST priority element (always at front)
// Time: O(1) dequeue, O(n) for next find (already sorted)
int pqDequeue(PriorityQueue* pq) {
    if (pq->size == 0) { printf("Priority Queue is empty!\n"); return -1; }

    int value = pq->items[0].value;
    // Shift all elements left
    for (int i = 0; i < pq->size - 1; i++)
        pq->items[i] = pq->items[i + 1];
    pq->size--;

    printf("PQ Dequeued: %d (highest priority)\n", value);
    return value;
}

// ——— DISPLAY PRIORITY QUEUE ———
void pqDisplay(PriorityQueue* pq) {
    printf("Priority Queue [high → low priority]:\n");
    for (int i = 0; i < pq->size; i++)
        printf("  [val=%d, pri=%d]\n", pq->items[i].value, pq->items[i].priority);
}
```

```
int main() {
    PriorityQueue pq;
    pqInit(&pq);

    printf("=== Priority Queue Demo ===\n\n");
    pqEnqueue(&pq, 100, 2);    // Normal priority
    pqEnqueue(&pq, 200, 5);    // Highest priority
    pqEnqueue(&pq, 300, 1);    // Lowest priority
    pqEnqueue(&pq, 400, 3);    // Medium priority

    pqDisplay(&pq);

    printf("\nServing elements in priority order:\n");
    pqDequeue(&pq);    // 200 (priority 5)
    pqDequeue(&pq);    // 400 (priority 3)
    pqDequeue(&pq);    // 100 (priority 2)
    pqDequeue(&pq);    // 300 (priority 1)

    return 0;
}
```

Queue Using Linked List

For a fully dynamic queue with no size limitation:


```
#include <stdio.h>
#include <stdlib.h>

struct QNode {
    int data;
    QNode* next;
};

// Maintain both front and rear pointers for O(1) operations
QNode* qFront = NULL;
QNode* qRear = NULL;

// — LL ENQUEUE — Add to REAR —————
void llEnqueue(int value) {
    QNode* newNode = (QNode*)malloc(sizeof(QNode));
    newNode->data = value;
    newNode->next = NULL;

    if (qRear == NULL) {
        // Empty queue: new node is both front AND rear
        qFront = qRear = newNode;
    } else {
        qRear->next = newNode; // Link new node to rear
        qRear = newNode;      // Update rear
    }
    printf("LL Enqueued: %d\n", value);
}

// — LL DEQUEUE — Remove from FRONT —————
int llDequeue() {
    if (qFront == NULL) {
```

```
        printf("LL Queue is empty!\n");
        return -1;
    }
    QNode* temp = qFront;
    int value = qFront->data;
    qFront = qFront->next;

    if (qFront == NULL)    // List is now empty
        qRear = NULL;

    free(temp);
    printf("LL Dequeued: %d\n", value);
    return value;
}

// — LL DISPLAY —————
void llQueueDisplay() {
    if (qFront == NULL) { printf("LL Queue empty.\n"); return; }
    printf("LL Queue [FRONT → REAR]: ");
    QNode* temp = qFront;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

Real-World Application: Task Scheduler Simulation

```
#include <stdio.h>
#include <string.h>
#define SCHED_MAX 10

// Task structure
struct Task {
    char name[30];
    int duration;    // Time in milliseconds
};

struct TaskQueue {
    Task tasks[SCHED_MAX];
    int front, rear, count;
};

void tqInit(TaskQueue* q) { q->front = 0; q->rear = -1; q->count = 0; }
int tqEmpty(TaskQueue* q) { return q->count == 0; }

void addTask(TaskQueue* q, const char* name, int duration) {
    if (q->count == SCHED_MAX) { printf("Scheduler full!\n"); return; }
    q->rear++;
    strcpy(q->tasks[q->rear].name, name);
    q->tasks[q->rear].duration = duration;
    q->count++;
    printf("  Scheduled: %-15s (%dms)\n", name, duration);
}

void runNextTask(TaskQueue* q) {
```

```
    if (tqEmpty(q)) { printf("No tasks pending.\n"); return; }
    Task t = q->tasks[q->front++];
    q->count--;
    printf("    Running:    %-15s (%dms) ... DONE\n", t.name, t.duration);
}

int main() {
    TaskQueue scheduler;
    tqInit(&scheduler);

    printf("=== Task Scheduler (FIFO Queue) ===\n\n");
    printf("Adding tasks to queue:\n");
    addTask(&scheduler, "Load Database",    200);
    addTask(&scheduler, "Authenticate",    50);
    addTask(&scheduler, "Fetch User Data", 150);
    addTask(&scheduler, "Render Page",     80);
    addTask(&scheduler, "Send Response",   30);

    printf("\nExecuting tasks in FIFO order:\n");
    while (!tqEmpty(&scheduler)) {
        runNextTask(&scheduler);
    }

    printf("\nAll tasks complete!\n");
    return 0;
}
```

Output:

=== Task Scheduler (FIFO Queue) ===

Adding tasks to queue:

Scheduled: Load Database (200ms)

Scheduled: Authenticate (50ms)

Scheduled: Fetch User Data (150ms)

Scheduled: Render Page (80ms)

Scheduled: Send Response (30ms)

Executing tasks in FIFO order:

Running: Load Database (200ms) ... DONE

Running: Authenticate (50ms) ... DONE

Running: Fetch User Data (150ms) ... DONE

Running: Render Page (80ms) ... DONE

Running: Send Response (30ms) ... DONE

All tasks complete!

Real-World Use Cases for Queues

QUEUE REAL-WORLD APPLICATIONS

SIMPLE QUEUE:

- Print spooling (documents print in order received)
- CPU process scheduling (round-robin)
- Customer service call centers

CIRCULAR QUEUE:

- Circular buffer in audio/video streaming
- Traffic light controller
- Ring buffer in OS kernel

PRIORITY QUEUE:

- Emergency room patient triage
- Dijkstra's shortest path algorithm
- A* pathfinding in games
- OS interrupt handling

DEQUE:

- Sliding window maximum problem
- Palindrome checking
- Undo/redo with both ends accessible



Common Mistakes with Queues

- ✗ **MISTAKE 1: False Full Problem (Simple Queue)**
Not using circular queue when elements are dequeued frequently.
FIX: Use circular queue or linked list queue.
- ✗ **MISTAKE 2: Off-by-one in rear initialization**
rear = 0 // WRONG: implies data[0] is occupied
rear = -1 // CORRECT: -1 means nothing enqueued yet
- ✗ **MISTAKE 3: Forgetting to update BOTH front and rear when queue becomes empty after a dequeue**
if (front == rear + 1) { front = 0; rear = -1; } // Reset
- ✗ **MISTAKE 4: Not checking isFull() before enqueue OR isEmpty() before dequeue – always validate first.**



Section 9.4 — Exercises

Exercise 1 (Beginner): Build a simple queue, enqueue the values 5, 10, 15, 20, 25, then dequeue all and print them. Observe the FIFO order.

Exercise 2 (Intermediate): Implement a circular queue of size 4. Enqueue 4 elements, dequeue 2, then enqueue 2 more. Show how the wrap-around works.

Exercise 3 (Advanced): Implement a queue using **two stacks**. (Hint: Use one stack for enqueue and one for dequeue. When the dequeue stack is empty, pop all from the enqueue stack into it.)



Section 9.4 Quick Reference

QUEUE OPERATIONS CHEAT SHEET		
Operation	Description	Time
enqueue(q, v)	Add v to REAR	O(1)
dequeue(q)	Remove & return from FRONT	O(1)
peek(q)	Return FRONT element (no remove)	O(1)
isEmpty(q)	Check if count == 0	O(1)
isFull(q)	Check if count == MAX	O(1)
Circular fix: $\text{index} = (\text{index} + 1) \% \text{MAX_SIZE}$		
KEY RULE: FIFO – First In, First Out		
FRONT dequeues, REAR enqueues		



3-Day Study Plan Summary

3-DAY INTENSIVE PLAN

DAY 1 (4-6 hours)

Morning: Read 9.0 and 9.1 thoroughly

Afternoon: Code all three array examples by hand

Evening: Complete all three 9.1 exercises

DAY 2 (4-6 hours)

Morning: Read 9.2 (Linked Lists) and type out code

Afternoon: Read 9.3 (Stacks), implement both versions

Evening: Complete exercises 9.2 and 9.3

DAY 3 (4-6 hours)

Morning: Read 9.4 (Queues), implement all 4 types

Afternoon: Build the Task Scheduler application

Evening: Complete 9.4 exercises; review everything



Master Cheat Sheet — All Data Structures

Structure	Access	Search	Insert	Key Rule
Array	$O(1)$	$O(n)$	$O(n)$	Index-based
Singly LL	$O(n)$	$O(n)$	$O(1)^*$	Pointer chain
Doubly LL	$O(n)$	$O(n)$	$O(1)^*$	Two pointers
Stack	$O(1)^{**}$	$O(n)$	$O(1)$	LIFO
Queue	$O(1)^{**}$	$O(n)$	$O(1)$	FIFO
Circular Q	$O(1)^{**}$	$O(n)$	$O(1)$	FIFO + wrap
Priority Q	$O(1)^{**}$	$O(n)$	$O(n)$	Priority first

* At head/tail with pointer; ** Only the accessible end (top/front)



Additional Resources

Books

- **"Data Structures and Algorithm Analysis in C++" by Mark Weiss** — The gold standard textbook. Comprehensive and rigorous.
- **"The C Programming Language" by Kernighan & Ritchie** — Master C-style coding from the inventors of C.
- **"Algorithms" by Sedgewick & Wayne** — Beautiful visualizations and deep intuition for every structure.
- **"Cracking the Coding Interview" by Gayle McDowell** — Practical application of every structure for interviews.

Online Platforms

- [Visualgo.net](#) — Animated visualizations of every data structure operation (free, browser-based)
- [LeetCode.com](#) — Practice problems organized by data structure (start with "Easy")
- [GeeksforGeeks.org](#) — Comprehensive articles and code for every concept in this guide
- [cppreference.com](#) — Official C++ reference documentation

Compiler & Practice

- **GCC (g++)** — Free C++ compiler available on all platforms
- [OnlineGDB.com](#) — Free online IDE for quick code testing
- [Repl.it](#) — Online coding environment, good for saving progress

Video Learning

- **MIT OpenCourseWare 6.006** — Introduction to Algorithms (free, world-class)
- **CS50 by Harvard (edX)** — Best beginner-friendly course covering C foundations

 **Final Pro Tip:** The best way to learn data structures is to implement them **by hand, from memory, without looking at notes**. Once you can write a linked list or stack from scratch in 10 minutes, you truly own the concept. Start with display → search → insert → delete for each structure, and you'll master them all.

End of Chapter 9: Data Structures and Operations

Guide Version 1.0 | C++ in C Style | Beginner to Expert